Prof. Dr. Stefan Leutenegger
Teaching Assistants: Tianyi Zhang, Cédric Le Gentil, and Simon Boche

Solutions to Exercise 6: Nonlinear Least Squares (NLSQ) Optimization

Session date: 14/04/2026

Task 1: Polynomial Fitting and Gauss-Newton Optimization (Python)

Consider the M^{th} order polynomial in the parameter p:

$$z = \sum_{j=0}^{M} a_j p^j,$$

where you are supposed to estimate the coefficients a_j, i.e. $\mathbf{x} = [a_0, a_1, \ldots, a_M]^T \in \mathbb{R}^{M+1}$. We are given a list of N measurements $\tilde{\mathbf{z}} = [\tilde{z}_1, \tilde{z}_2, \ldots, \tilde{z}_N]^T$ and corresponding parameters $\tilde{\mathbf{p}} = [\tilde{p}_1, \tilde{p}_2, \ldots, \tilde{p}_N]$.

i) With the sum of square errors:

$$c(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^{N} \|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|^2, \tag{1}$$

write down the solution for $\mathbf{x}$ that minimises it.

As in Lecture 6, Slide 13, we see that we can re-write the above as

$$c(\mathbf{x}) = \frac{1}{2} (\tilde{\mathbf{z}} - \mathbf{H}\mathbf{x})^T (\tilde{\mathbf{z}} - \mathbf{H}\mathbf{x}),$$

with

$$\mathbf{H} = \begin{bmatrix} 1 & p_1 & p_1^2 & \ldots & p_1^M \\ 1 & p_2 & p_2^2 & \ldots & p_2^M \\ \vdots & \vdots & \vdots & \ldots & \vdots \\ 1 & p_N & p_N^2 & \ldots & p_N^M \end{bmatrix}.$$

Now, the normal equations can directly be applied as

$$\mathbf{x}^* = (\mathbf{H}^T\mathbf{H})^{-1}\mathbf{H}^T\tilde{\mathbf{z}}.$$

ii) (Python) Implement the above scheme in `e06_least_squares.py`, where the measurements are already provided, and where the implementation should work for any $N > M$.

See `e06_least_squares_sol.py`.

iii) (Python) Unfortunately, we now have some outlier measurements. Change the outlier probability parameter `P_outlier` to a value > 0, and observe the effect on your solution.

See `e06_least_squares_sol.py`.

iv) (**Advanced**, Python) To accommodate for outliers, we introduce the alternative of a robust cost function:

$$c(\mathbf{x}) = \sum_{i=1}^{N} \rho\left(\|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|\right),$$

where $\rho(.)$ stands for a robust cost function. Propose a new implementation to estimate $\mathbf{x}$ using Gauss-Newton. For initialisation you may still use the direct Least Squares Solution from above.

Background: The introduction of a robust cost function makes the error terms of (1) nonlinear, which is why we need to use iterative application of the Normal Equations, i.e. the Gauss-Newton scheme. In order to follow the recipe from Lecture 6, we introduce the error terms $e_i = \tilde{z}_i - h_i(\mathbf{x})$, where of course we are still dealing with a linear measurement function here, i.e. $h_i(\mathbf{x}) = \mathbf{H}_i\mathbf{x}$, with

$$\mathbf{H}_i = \begin{bmatrix} 1 & p_i & p_i^2 & \dots & p_i^M \end{bmatrix},$$

therefore we also get the Jacobian of the error term as $\mathbf{E}_i = -\mathbf{H}_i$.

Now for the robustification, we introduce different functions $\rho(r)$, with $r = ||\mathbf{e}_i||$. As introduced in the lecture, we can now re-write the problem as

$$c(\mathbf{x}) = \sum_{i=1}^{N} \rho\left(\|\mathbf{e}_i\|\right) = \frac{1}{2}\sum_{i=1}^{N} \|\mathbf{e}_i'\|^2,$$

with $\mathbf{e}_i' = w(r)\mathbf{e}_i$, and $w(r) = \sqrt{2\rho(r)}/r$.

Typical choices for the robustifier are the Huber Loss ρ_H or the Cauchy Loss ρ_C:

$$\rho_{\mathrm{H}} = \begin{cases} \frac{1}{2}r^2, & r \le k, \\ k|r| - \frac{1}{2}k^2, & r > k, \end{cases} \qquad \rho_{\mathrm{C}} = \frac{c^2}{2}\log\left(1 + \frac{r^2}{c^2}\right),$$

with additional tunable parameters k and c. **Hint:** You will have to derive the new Jacobians $\mathbf{E}_i'$.

To get the Jacobian of $\mathbf{e}_i'$, we therefore have to apply the product rule as follows:

$$\mathbf{E}_i' = \frac{\partial \mathbf{e}_i'}{\partial \mathbf{x}} = \frac{\partial w(r(\mathbf{e}_i(x)))}{\partial \mathbf{x}} \mathbf{e}_i + w(r)\mathbf{E}_i.$$

We observe that we can readily evaluate the second addend, but need to compute the first one with extensive use of the chain rule:

$$\frac{\partial w(r(\mathbf{e}_i(x)))}{\partial \mathbf{x}} = \frac{\partial w(r)}{\partial r} \frac{\partial r(\mathbf{e}_i)}{\partial \mathbf{e}_i} \mathbf{E}_i.$$

The Jacobian of the norm is found as $\frac{r(\mathbf{e}_i)}{\mathbf{e}_i} = \frac{\mathbf{e}_i^T}{r}$. We will now just need to now plug in the different cases of $\rho(r)$ and compute the remaining derivative.
Huber: we have the robustifier

$$\rho_{\mathrm{H}} = \begin{cases} \frac{1}{2}r^2, & r \leq k, \\ k|r| - \frac{1}{2}k^2, & r > k, \end{cases}$$

thus we see that we get as expected $w(r) = 1$ and thus $\mathbf{E}_i' = \mathbf{E}_i$ for $r \leq k$; for the second case (of large error terms), however, we get

$$w(r) = \frac{\sqrt{2k|r| - k^2}}{r}$$

and thus we get the remaining term (noting that $r \geq 0$, too)

$$\frac{\partial w(r)}{\partial r} = \frac{k}{r\sqrt{2kr - k^2}} - \frac{\sqrt{-k^2 + 2rk}}{r^2}.$$

Cauchy: we have the robustifier

$$\rho_{\mathrm{C}} = \frac{c^2}{2} \log\left(1 + \frac{r^2}{c^2}\right).$$

Therefore, we get

$$w(r) = \frac{\sqrt{c^2 \log\left(1 + \frac{r^2}{c^2}\right)}}{r}$$

and thus we get the remaining term

$$w'(r) := \frac{\partial w(r)}{\partial r} = \frac{1}{\left(\frac{r^2}{c^2} + 1\right)\sqrt{c^2 \log\left(\frac{r^2}{c^2} + 1\right)}} - \frac{\sqrt{c^2 \log\left(\frac{r^2}{c^2} + 1\right)}}{r^2}.$$

Here, $w(0)$ and $w'(0)$ are obviously not defined. However, we find that $\lim_{r \downarrow 0} w(r) = 1$, $\lim_{r \downarrow 0} w'(r) = 0$, and also have $\rho(0) = 1$. Note that both of these have to be the case, because we want the robustified error terms to still behave like regular squared error terms for small error magnitudes r. But here we need to pay special attention in the implementation regarding numerics, when r gets very small.

Regarding the implementation of all of the above, see `e06_least_squares_sol.py`.

Task 2: 2D Motion Tracking System (Python)

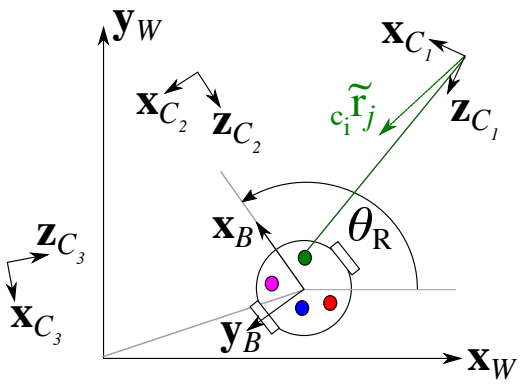

Figure 1: A two-wheel differential drive mobile robot with four fixed markers (indicated by the colorful dots on the robot base) observed by three fixed cameras measuring the bearing vectors ${}_{C_i}\tilde{\mathbf{r}}_j$.

Consider the two-wheel differential drive mobile robot shown in Figure 1 with the following state:

$$\mathbf{x}^T = [x_R, y_R, \theta_R]^T,$$

where x_R, y_R denote the 2D position of the robot base frame in the world frame, and θ is the orientation of the robot frame relative to the world frame. The robot is equipped with 4 distinct markers (index j) at positions ${}_B\mathbf{t}_{M_j}$ (note the expression in the robot body coordinate frame $\underrightarrow{\mathcal{F}}_B$). Now there are three fixed cameras (index i) located at poses $\boldsymbol{T}_{WC_i}$ that measure the bearing vectors (lengths normalised to 1) ${}_{C_i}\tilde{\mathbf{r}}_j$ to those markers (2D in the x-y plane). Careful: the bearings are measured relative to the respective camera's individual coordinate frame.

i) (Python) Open the Python script `e06_diff_drive_mocap.py` to see the set-up: the simulation will show the robot carrying out a 2D motion in a room.

> See `e06_diff_drive_mocap_sol.py`.

ii) Formulate the angular difference $e_{i,j}$ between the measured bearing (${}_{C_i}\tilde{\mathbf{r}}_j$) and the actual bearing to the marker j (${}_{C_i}\mathbf{t}_{M_j}$) which can be expressed as a function of the state and the known positions of the marker in the body frame, ${}_{B}\mathbf{t}_{M_j}$. Also write down the respective sum of squares cost function $c(\mathbf{x})$.

There are many ways to formulate the bearing angle difference, leading to different levels of difficulties in deriving what follows. We chose to do it as:

$$e_{i,j} = \arctan\left(\frac{[1,0]_{C_i}\tilde{\mathbf{r}}_j}{[0,1]_{C_i}\tilde{\mathbf{r}}_j}\right) - \arctan\left(\frac{[1,0,0]_{C_i}\mathbf{t}_{M_j}(\mathbf{x})}{[0,0,1]_{C_i}\mathbf{t}_{M_j}(\mathbf{x})}\right),$$

where ${}_{C_i}\mathbf{t}_{M_j} =: [t_1, t_2, t_3]^T$ is the j^{th} marker expressed in the i^{th} camera as

$${}_{C_i}\boldsymbol{t}_{M_j} = \boldsymbol{T}_{C_iW}\boldsymbol{T}_{WB}\boldsymbol{t}_{M_j} = \begin{bmatrix} \mathbf{R}_{WC_i}{}^T & -\mathbf{R}_{WC_i}{}^T {}_W\mathbf{t}_{C_i} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{WB} & {}_W\mathbf{t}_B \\ \mathbf{0} & 1 \end{bmatrix} {}_B\boldsymbol{t}_{M_j},$$

multiplied out

$${}_{C_i}\mathbf{t}_{M_j} = \mathbf{R}_{WC_i}{}^T((\mathbf{R}_{WB}\,{}_B\mathbf{t}_{M_j} + {}_W\mathbf{t}_B) - {}_W\mathbf{t}_{C_i}), \tag{2}$$

where we introduced the robot pose in terms of position ${}_W\mathbf{t}_B = [x_R, y_R, 0]^T$ and orientation $\mathbf{R}_{WB} = \mathbf{R}_z(\theta_R)$, whereby

$$\mathbf{R}_z(\theta_R) = \begin{bmatrix} \cos\theta_R & -\sin\theta_R & 0 \\ \sin\theta_R & \cos\theta_R & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Now we can simply formulate the cost to be minimised as

$$c(\mathbf{x}) = \frac{1}{2}\sum_{i,j} e_{i,j}^2.$$

iii) Derive the Jacobian $\mathbf{E}_{i,j} = \frac{\partial e_{i,j}}{\partial \mathbf{x}}$.

We can compute the Jacobian as always using the chain rule. Introducing $a({}_{C_i}\mathbf{t}_{M_j}(\mathbf{x})) = \frac{t_1}{t_3}$, we now obtain

$$\mathbf{E}_{i,j} = \frac{\partial e_{i,j}}{\partial \mathbf{x}} = -\frac{\partial \arctan(a)}{\partial a}\frac{\partial a}{\partial {}_{C_i}\mathbf{t}_{M_j}}\frac{\partial {}_{C_i}\mathbf{t}_{M_j}}{\mathbf{x}}.$$

We can look up

$$\frac{\partial \arctan(a)}{\partial a} = \frac{1}{a^2+1}.$$

The term $\frac{\partial a}{\partial {}_{C_i}\mathbf{t}_{M_j}}$ really is something like a 2D equivalent of a camera projection into the unit plane (unit line here), so we get a somewhat familiar term:

$$\frac{\partial a}{\partial {}_{C_i}\mathbf{t}_{M_j}} = \begin{bmatrix} \frac{1}{\bar{t}_3} & 0 & -\frac{\bar{t}_1}{\bar{t}_3^2} \end{bmatrix}.$$

The perhaps trickiest part is coming now regarding the position/orientation:

$$\frac{\partial {}_{C_i}\mathbf{t}_{M_j}}{x_R} = {\mathbf{R}_{WC_i}}^T \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \frac{\partial {}_{C_i}\mathbf{t}_{M_j}}{y_R} = {\mathbf{R}_{WC_i}}^T \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},$$

$$\begin{aligned} \frac{\partial {}_{C_i}\mathbf{t}_{M_j}}{\theta_R} &= {\mathbf{R}_{WC_i}}^T \frac{\partial}{\partial \theta_R}\left(\begin{bmatrix} \cos\theta_R & -\sin\theta_R & 0 \\ \sin\theta_R & \cos\theta_R & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{M_j} \\ y_{M_j} \\ 0 \end{bmatrix} \right) \\ &= {\mathbf{R}_{WC_i}}^T \begin{bmatrix} -\sin\bar{\theta}_R x_{M_j} - \cos\bar{\theta}_R y_{M_j} \\ \cos\bar{\theta}_R x_{M_j} - \sin\bar{\theta}_R y_{M_j} \\ 0 \end{bmatrix}. \end{aligned}$$

iv) (Python) Using the Gauss-Newton optimisation scheme, implement an estimator for the robot state $\mathbf{x}$ that minimises the above cost $c(\mathbf{x})$ in `e06_diff_drive_mocap.py` .

See `e06_diff_drive_mocap_sol.py`.